

Table of Contents

Part 1 — DSA Templates

Binary Search (2 cards)
Upper Middle (even → take upper)
Lower Middle (even → take lower)

Bit Manipulation (5 cards)
Sum of Submasks (SOS DP)
Sum of Supermasks
Largest Bit Set
Lowest Bit Set
Enumerate All Submasks of a Mask

Euler Path (1 cards)
Hierholzer's Algorithm

Fenwick Tree (BIT) (1 cards)
Binary Indexed Tree — 1-indexed

Graph Theory (9 cards)
Preorder Traversal
Inorder Traversal
Postorder Traversal
BFS (Breadth First Search)
Bellman-Ford Algorithm
Dijkstra's Algorithm
Topological Sort (BFS)
Kahn's Algorithm
Disjoint Set Union (DSU / Union-Find)

Linked List (1 cards)
Floyd's Fast & Slow Pointer

Manacher's Algorithm (1 cards)
Palindromic Substrings — All Centers

Ordered Set (PBDS) (1 cards)
GNU C++ Policy-Based Ordered Set

Prefix & Suffix Sum (4 cards)
Prefix Sum (starts at index 0)
Prefix Sum (1-indexed, starts at 0)
Suffix Sum (starts at last index)
Suffix Sum (1-indexed, starts at 0)

Segment Tree (1 cards)
Segment Tree — Range Sum + Point Update

Sparse Table (1 cards)
Sparse Table — O(1) Range Queries

Two Pointers (1 cards)
Two Pointers — Converging

Part 2 — Tutorials

Basic Topics

Arrays (1)
Backtracking (1)
Binary Search (1)
Brute Force (1)
Greedy (1)
Hash Map (1)
Heap / Priority Queue (1)
Kadane's Algorithm (1)
Linear Search (1)
Linked List (1)
MOD (1e9 + 7) (1)
Prefix Sum (1)
Queue & Stack (2)
Sliding Window (2)
Sorting (4)
Time Complexity (1)
Trie (1)
Two Pointers (1)

Graph Theory

Tree Traversals (1)
BFS (1)
DFS (1)
Bellman-Ford (1)
Dijkstra's (1)
DSU (1)
Topological Sort (1)
Binary Search Tree (1)
Kruskal's MST (1)
Tarjan's Algorithm (1)
LCA (1)

Math

Bit Manipulation (2)
Combinatorics (1)
Matrix Exponentiation (1)
Number Theory (2)
Prime Factors (2)

Strings

Manacher's Algorithm (1)
Palindrome (1)
Z Algorithm (1)

Part 1 — DSA Templates

Binary Search · 2 cards

Upper Middle (even → take upper)

For even-length arrays, takes the upper middle element.

```
C++
int search(vector<int>& nums, int target) {
    int n = nums.size(), l = 0, r = n - 1;
    while (l < r) {
        // for even number of elements, take the upper one
        int m = l + (r - l + 1) / 2;
        if (target < nums[m]) r = m - 1;
        else l = m;
    }
    return nums[l] == target ? l : -1;
}
```

```
Java
int search(int[] nums, int target) {
    int n = nums.length, l = 0, r = n - 1;
    while (l < r) {
        // for even number of elements, take the upper one
        int m = l + (r - l + 1) / 2;
        if (target < nums[m]) r = m - 1;
        else l = m;
    }
    return nums[l] == target ? l : -1;
}
```

Lower Middle (even → take lower)

For even-length arrays, takes the lower middle element.

```
C++
int search(vector<int>& nums, int target) {
    int n = nums.size(), l = 0, r = n - 1;
    while (l < r) {
        // for even number of elements, take the lower one
        int m = l + (r - l) / 2;
        if (target > nums[m]) l = m + 1;
        else r = m;
    }
    return nums[l] == target ? l : -1;
}
```

```
Java
int search(int[] nums, int target) {
    int n = nums.length, l = 0, r = n - 1;
    while (l < r) {
        // for even number of elements, take the lower one
        int m = l + (r - l) / 2;
        if (target > nums[m]) l = m + 1;
        else r = m;
    }
    return nums[l] == target ? l : -1;
}
```

Bit Manipulation · 5 cards

Sum of Submasks (SOS DP)

```
C++
template<typename T_out, typename T_in>
vector<T_out> submask_sums(int n, const vector<T_in> &values) {
    assert(int(values.size()) == 1 << n);
    vector<T_out> dp(values.begin(), values.end());
    for (int i = 0; i < n; i++)
        for (int base = 0; base < 1 << n; base += 1 << (i + 1))
            for (int mask = base; mask < base + (1 << i); mask++)
                dp[mask + (1 << i)] += dp[mask];
    return dp;
}
```

Sum of Supermasks

```
C++
template<typename T_out, typename T_in>
vector<T_out> supermask_sums(int n, vector<T_in> values) {
    reverse(values.begin(), values.end());
    vector<T_out> result = submask_sums<T_out>(n, values);
    reverse(result.begin(), result.end());
    return result;
}
```

Largest Bit Set

```
C++
int largest_bit(int x) {
    return x == 0 ? -1 : 31 - __builtin_clz(x);
}
```

Lowest Bit Set

```
C++
int lowest_bit(int x) {
    return x & (-x);
}

// Index of lowest bit (1-based)
int lowest_bit_index(int x) {
    return __builtin_ffs(x);
}
```

Enumerate All Submasks of a Mask

```
C++
for (int s = m; s; s = (s - 1) & m) {
    // use s
}
```

Euler Path · 1 card

Hierholzer's Algorithm

Finds Euler path/circuit in a directed graph. Used in Reconstruct Itinerary (LC 332).

```
C++
void euler(unordered_map<string, queue<string>& g,
          string src, vector<string>& ans) {
    while (!g[src].empty()) {
        string nxt = g[src].front();
        g[src].pop();
        euler(g, nxt, ans);
    }
    ans.push_back(src);
}

// Usage (LC 332 - Reconstruct Itinerary)
vector<string> findItinerary(vector<vector<string>>& tickets) {
    vector<string> ans;
    sort(tickets.begin(), tickets.end(),
         [](const vector<string>& x, const vector<string>& y) {
             return x[1] < y[1];
         });
    unordered_map<string, queue<string>> g;
    for (auto x : tickets) g[x[0]].push(x[1]);
    euler(g, "JFK", ans);
    reverse(ans.begin(), ans.end());
    return ans;
}
```

Fenwick Tree (BIT) · 1 card

Binary Indexed Tree — 1-indexed

Supports point update and range sum query in $O(\log n)$. Also supports range update via difference array trick.

```
C++
template <class T>
struct BIT { // 1-indexed
    int n;
    vector<T> t;

    BIT() {}
    BIT(int _n) { n = _n; t.assign(n + 1, 0); }

    T query(int i) {
        T ans = 0;
        for (; i >= 1; i -= (i & -i)) ans += t[i];
        return ans;
    }

    void upd(int i, T val) {
        if (i <= 0) return;
        for (; i <= n; i += (i & -i)) t[i] += val;
    }

    // Range update [l, r]
    void upd(int l, int r, T val) {
        upd(l, val);
        upd(r + 1, -val);
    }

    // Range query [l, r]
    T query(int l, int r) {
        return query(r) - query(l - 1);
    }
};
// BIT->int> bit(n);
```

Graph Theory · 9 cards

Preorder Traversal

Visit root → left → right.

```
C++
void preorder(TreeNode* node) {
    if (node == NULL) return;
    // process node->val here
    preorder(node->left);
    preorder(node->right);
}
```

```
Python
def preorder(node):
    if node is None: return
    # process node.val here
    preorder(node.left)
    preorder(node.right)
```

Inorder Traversal

Visit left → root → right. Gives sorted order for BSTs.

```
C++
void inorder(TreeNode* node) {
    if (node == NULL) return;
    inorder(node->left);
    // process node->val here
    inorder(node->right);
}
```

```
Python
def inorder(node):
    if node is None: return
    inorder(node.left)
    # process node.val here
    inorder(node.right)
```

Postorder Traversal

Visit left → right → root.

```
C++
void postorder(TreeNode* node) {
    if (node == NULL) return;
    postorder(node->left);
    postorder(node->right);
    // process node->val here
}
```

Python

```
def postorder(node):
    if node is None: return
    postorder(node.left)
    postorder(node.right)
    # process node.val here
```

BFS (Breadth First Search)

Explore all closest nodes first before going deeper.

```
Python
def bfs(root, targetValue):
    if root is None: return None
    currentLevel = [root]
    while len(currentLevel) > 0:
        nextLevel = []
        for node in currentLevel:
            if node is None: continue
            if node.val == targetValue: return node
            nextLevel.append(node.left)
            nextLevel.append(node.right)
        currentLevel = nextLevel
    return None
```

Bellman-Ford Algorithm

Shortest paths from single source. Handles negative weights.

```
C++
template<typename T, &T>
void bellman_ford(T &g, T_vector &dist, int src, int mx_edges) {
    dist[src] = 0;
    for (int i = 0; i < mx_edges; i++) {
        T_vector ndist = dist;
        for (auto x : g) {
            auto (from, to, cost) = x;
            ndist[to] = min(ndist[to], dist[from] + cost);
        }
        dist = ndist;
    }
}
```

Dijkstra's Algorithm

Shortest paths from single source. Non-negative weights only.

```

C++
template<typename T_pair, typename T_vector>
void dijkstra(T_pair &g, T_vector &dist, int start) {
    priority_queue<pair<int, int>, vector<pair<int, int>>,
        greater<pair<int, int>>> pq;

    dist[start] = 0;
    pq.push(start, 0);
    while (!pq.empty()) {
        auto [u_node, u_cost] = pq.top(); pq.pop();
        if (u_cost > dist[u_node]) continue;
        for (auto [v_node, v_cost] : g[u_node]) {
            if (dist[v_node] > dist[u_node] + v_cost) {
                dist[v_node] = dist[u_node] + v_cost;
                pq.push(v_node, dist[v_node]);
            }
        }
    }
}
}

```

Topological Sort (BFS)

Linear ordering of vertices in a DAG.

```

C++
struct TopologicalSort {
    int n, steps = 0, nodes = 0;
    vector<int> indegree, orders;
    vector<vector<int>> G;
    bool isTopologicalSorted = false;

    TopologicalSort(vector<vector<int>>& g, vector<int>& in) {
        G = g; n = G.size(); indegree = in;
        queue<int> q;
        for (int i = 0; i < n; i++)
            if (indegree[i] == 0) q.push(i);
        while (!q.empty()) {
            int sz = q.size();
            steps++; nodes += sz;
            for (int i = 0; i < sz; i++) {
                auto u = q.front(); q.pop();
                orders.push_back(u);
                for (auto v : G[u])
                    if (--indegree[v] == 0) q.push(v);
            }
        }
        isTopologicalSorted = nodes == n;
    }
};

```

Kahn's Algorithm

Alternative topological sort. Returns empty if cycle detected.

```

C++
template<typename T_vector, typename T_vector_vector>
T_vector kahn(int n, T_vector_vector &edges) {
    vector<int> ordering, indegree(n, 0);
    vector<vector<int>> g(n);
    for (auto e : edges) {
        --e[0]; --e[1];
        indegree[e[1]]++;
        g[e[0]].push_back(e[1]);
    }
    queue<int> q;
    for (int i = 0; i < n; i++)
        if (indegree[i] == 0) q.push(i);
    int visited = 0;
    while (!q.empty()) {
        int u = q.front(); q.pop();
        ordering.push_back(u);
        visited++;
        for (auto v : g[u])
            if (--indegree[v] == 0) q.push(v);
    }
    if (visited != n) return T_vector{};
    reverse(ordering.begin(), ordering.end());
    return ordering;
}

```

Disjoint Set Union (DSU / Union-Find)

Combines sets and checks connectivity in near O(1) per operation.

```

C++
class dsu {
public:
    vector<int> root, rank;
    int n, cnt;

    dsu(int _n) : n(_n) {
        root.resize(n); rank.resize(n);
        for (int i = 0; i < n; i++) { root[i] = i; rank[i] = 1; }
        cnt = n;
    }

    int getCount() { return cnt; }

    int get(int x) {
        return x == root[x] ? x : (root[x] = get(root[x]));
    }

    bool unite(int x, int y) {
        x = get(x); y = get(y);
        if (x == y) return false;
        if (rank[x] > rank[y]) root[y] = x;
        else if (rank[x] < rank[y]) root[x] = y;
        else { root[y] = x; rank[x]++; }
        cnt--;
        return true;
    }
};

```

Linked List · 1 card

Floyd's Fast & Slow Pointer

Detects cycles and finds midpoints in linked lists.

```

C++
ListNode slow = head;
ListNode fast = head;
while (fast != nullptr && fast->next != nullptr) {
    // do something here
    slow = slow->next;
    fast = fast->next->next;
}

```

```

Java
ListNode slow = head, fast = head;
while (fast != null && fast.next != null) {
    // do something here
    slow = slow.next;
    fast = fast.next.next;
}

```

Manacher's Algorithm · 1 card

Palindromic Substrings — All Centers

Finds all palindromic substrings in O(n). Converts even/odd cases by inserting # separators.

```

Python
def palindromicSubstrings(s: str) -> list[str]:
    if not s: return []
    # Insert # to handle even-length palindromes
    string = "#".join(s)
    plengths = [0] * len(string)
    c = R = 0
    for i in range(len(string)):
        if i < R:
            plengths[i] = min(R - i, plengths[2 * c - i])
            while (i - plengths[i] - 1) != 0 and
                i + plengths[i] + 1 < len(string) and
                    string[i - plengths[i] + 1] == string[i + plengths[i] + 1]):
                plengths[i] += 1
            if plengths[i] + 1 > R:
                c, R = i, i + plengths[i]
        result = []
        for i, p in enumerate(plengths):
            s = string[i - p // 2 : i + p // 2 + 1].replace("#", "")
            if s: result.append(s)
    return result

```

```

C++
vector<string> palindromicSubstrings(string s) {
    if (s.empty()) return {};
    string str = "";
    for (int i = 0; i < (int)s.length() - 1; i++) {
        str += s[i]; str += "#";
    }
    str += s.back();
    int len = str.length();
    vector<int> pl(len, 0);
    int c = 0, R = 0;
    for (int i = 0; i < len; i++) {
        if (i < R) pl[i] = min(R - i, pl[2 * c - i]);
        while (i - pl[i] - 1 == 0 && i + pl[i] + 1 < len &&
            str[i - pl[i] + 1] == str[i - pl[i] - 1])
            pl[i]++;
        if (pl[i] + 1 > R) { c = i; R = i + pl[i]; }
    }
    vector<string> res;
    for (int i = 0; i < len; i++) {
        string p = str.substr(i - pl[i], 2 * pl[i] + 1);
        string r = "";
        for (char ch : p) if (ch != "#") r += ch;
        res.push_back(r);
    }
    return res;
}

```

Ordered Set (PBDS) · 1 card

GNU C++ Policy-Based Ordered Set

O(log n) rank queries: order_of_key(k) = # elements < k, find_by_order(k) = k-th element.

```

C++
#include <ext/pb_ds/assoc_container.hpp>
#include <ext/pb_ds/tree_policy.hpp>
using namespace __gnu_pbds;

// less<int> -> ascending order set
// greater<int> -> descending order set
// less_equal<int> -> allows duplicates (multiset)
tree<int, null_type, less<int>, rb_tree_tag,
    tree_order_statistics_node_update> oset;

void usage() {
    oset.insert(5);
    // # elements strictly less than k
    int rank = oset.order_of_key(5);
    // k-th element (0-indexed)
    auto it = oset.find_by_order(0);
}

```

Prefix & Suffix Sum · 4 cards

Prefix Sum (starts at index 0)

pref[i] = sum of a[0..i]

```
C++
vector<int> pref(n);
pref[0] = a[0];
for (int i = 1; i < n; i++)
    pref[i] = pref[i - 1] + a[i];

// Query sum [l, r]:
// l == 0 ? pref[r] : pref[r] - pref[l - 1]
```

Prefix Sum (1-indexed, starts at 0)

pref[i+1] = sum of a[0..i]. Clean range queries.

```
C++
vector<int> pref(n + 1, 0);
for (int i = 0; i < n; i++)
    pref[i + 1] = pref[i] + a[i];

// Query sum [l, r] (0-indexed):
// pref[r + 1] - pref[l]
```

Suffix Sum (starts at last index)

suff[i] = sum of a[i..n-1]

```
C++
vector<int> suff(n);
suff[n - 1] = a[n - 1];
for (int i = n - 2; i >= 0; i--)
    suff[i] = suff[i + 1] + a[i];
```

Suffix Sum (1-indexed, starts at 0)

suff[i] = sum of a[i..n-1] with extra 0 at end.

```
C++
vector<int> suff(n + 1, 0);
for (int i = n - 1; i >= 0; i--)
    suff[i] = suff[i + 1] + a[i];

// Query sum [l, r] (0-indexed):
// suff[l] - suff[r + 1]
```

Segment Tree · 1 card

Segment Tree — Range Sum + Point Update

O(log n) point update and range sum query.

```
C++
struct segtree {
    vector<long long> sums;
    int size;

    void init(int n) {
        size = 1;
        while (size < n) size *= 2;
        sums.assign(size * 2, 0LL);
    }

    void set(int i, int v, int x, int lx, int rx) {
        if (lx == rx) { sums[x] = v; return; }
        int m = (lx + rx) / 2;
        if (i < m) set(i, v, 2*x+1, lx, m);
        else set(i, v, 2*x+2, m, rx);
        sums[x] = sums[2*x+1] + sums[2*x+2];
    }

    void set(int i, int v) { set(i, v, 0, 0, size); }

    long long sum(int l, int r, int x, int lx, int rx) {
        if (lx >= r || l >= rx) return 0;
        if (lx >= l && rx <= r) return sums[x];
        int m = (lx + rx) / 2;
        return sum(l, r, 2*x+1, lx, m) + sum(l, r, 2*x+2, m, rx);
    }

    long long sum(int l, int r) { return sum(l, r, 0, 0, size); }
} st;

// Usage:
// st.init(n);
// st.set(i, val); // point update
// st.sum(l, r + 1); // range sum [l, r] (exclusive r)
```

Sparse Table · 1 card

Sparse Table — O(1) Range Queries

Static arrays only (no updates). Ideal for range min/max/gcd in O(1) after O(n log n) build.

```
C++
template<typename it, typename bin_op>
struct sparse_table {
    using T = typename remove_reference<decltype(*declval<it>())>::type;
    vector<vector<T>>> t;
    bin_op f;

    sparse_table(it first, it last, bin_op op) : t{1}, f{op} {
        int n = distance(first, last);
        t.assign(32, _builtin_clz(n), vector<T>(n));
        t[0].assign(first, last);
        for (int i = 1; i < (int)t.size(); i++)
            for (int j = 0; j < n - (1 << i) + 1; j++)
                t[i][j] = f(t[i-1][j], t[i-1][j] + (1 << (i-1)));
    }

    // O(1) range query f(a[l..r])
    T query(int l, int r) {
        int h = floor(log2(r - l + 1));
        return f(t[h][l], t[h][r - (1 << h) + 1]);
    }
};

// Usage - range GCD:
// sparse_table g(a.begin(), a.end()), [](int x, int y){ return gcd(x, y); };
// g.query(l, r);
```

Two Pointers · 1 card

Two Pointers — Converging

l and r start at opposite ends and move toward each other.

```
C++
int f(vector<int>& v) {
    int ans = 0;
    int l = 0, r = (int)v.size() - 1;
    while (l < r) {
        // logic here
        if (/= condition *) l++;
        else r--;
    }
    return ans;
}
```

```
Java
int f(int[] v) {
    int ans = 0;
    int l = 0, r = v.length - 1;
    while (l < r) {
        // logic here
        if (/= condition *) l++;
        else r--;
    }
    return ans;
}
```

Part 2 — Tutorials

Basic Topics

Arrays · 1 card

Array Operations & Complexity

Arrays store collections in contiguous memory. Support O(1) access by index but O(n) insertion/deletion at arbitrary positions.

Access O(1) - Insert/Delete middle O(n) - Append O(1) amortized

```
C++
// Sort ascending
sort(v.begin(), v.end());
// Sort descending
sort(v.begin(), v.end(), greater<int>());
// Custom comparator
sort(v.begin(), v.end(), [](const auto& a, const auto& b) {
    return a[0] != b[0] ? a[0] < b[0] : a[1] < b[1];
});
// Max / min / sum
int mx = *max_element(v.begin(), v.end());
int mn = *min_element(v.begin(), v.end());
int s = accumulate(v.begin(), v.end(), 0);
```

```
Python
# Sort ascending / descending
arr.sort()
arr.sort(reverse=True)
arr.sort(key=lambda x: x[1]) # by second element

# Max / min / sum
mx = max(arr)
mn = min(arr)
s = sum(arr)
```

Backtracking · 1 card

Backtracking Template

Explores all possible solutions by building candidates incrementally and abandoning ("backtracking") invalid paths. Uses DFS recursion.

Time O(n!) worst case - Space O(n) recursion depth

```
C++
vector<vector<int>> ans;
vector<int> tmp;

void backtrack(vector<int>& nums, vector<bool>& used) {
    // Base case: solution complete
    if (tmp.size() == nums.size()) {
        ans.push_back(tmp);
        return;
    }
    for (int i = 0; i < nums.size(); i++) {
        if (used[i]) continue;
        // Choose
        used[i] = true;
        tmp.push_back(nums[i]);
        // Explore
        backtrack(nums, used);
        // Un-choose (backtrack)
        tmp.pop_back();
        used[i] = false;
    }
}

// LC 46: Permutations
vector<vector<int>> permute(vector<int>& nums) {
    vector<bool> used(nums.size(), false);
    backtrack(nums, used);
    return ans;
}
```

```
Python
def permute(nums):
    ans, tmp = [], []
    used = [False] * len(nums)

    def backtrack():
        if len(tmp) == len(nums):
            ans.append(tmp[:])
            return
        for i in range(len(nums)):
            if used[i]: continue
            used[i] = True
            tmp.append(nums[i])
            backtrack()
            tmp.pop()
            used[i] = False

    backtrack()
    return ans
```

Binary Search · 1 card

Standard Binary Search

Efficiently locates elements in a sorted array by halving the search space each iteration. Requires sorted input.

Time $O(\log n)$ · Space $O(1)$

```
C++
int binarySearch(vector<int>& nums, int target) {
    int lp = 0, rp = nums.size() - 1;
    while (lp <= rp) {
        int mid = lp + (rp - lp) / 2; // avoids overflow
        if (nums[mid] == target) return mid;
        else if (nums[mid] < target) lp = mid + 1;
        else rp = mid - 1;
    }
    return -1;
}
```

```
Python
def binarySearch(nums, target):
    lp, rp = 0, len(nums) - 1
    while lp <= rp:
        mid = (lp + rp) // 2
        if nums[mid] == target: return mid
        elif nums[mid] < target: lp = mid + 1
        else: rp = mid - 1
    return -1
```

```
Java
int binarySearch(int[] nums, int target) {
    int lp = 0, rp = nums.length - 1;
    while (lp <= rp) {
        int mid = lp + (rp - lp) / 2;
        if (nums[mid] == target) return mid;
        else if (nums[mid] < target) lp = mid + 1;
        else rp = mid - 1;
    }
    return -1;
}
```

Brute Force · 1 card

Brute Force — Nested Loops

Try every possible combination without optimization. Use as a baseline before optimizing. $O(n^2)$ or worse. Time $O(n^2)$ typical · Space $O(1)$

```
C++
// LC 455: Assign Cookies
// Strategy: match smallest sufficient cookie to least greedy child
int findContentChildren(vector<int>& g, vector<int>& s) {
    sort(s.begin(), s.end());
    sort(g.begin(), g.end());
    int ans = 0;
    for (int j = 0; j < s.size() && ans < g.size(); j++)
        if (g[ans] <= s[j]) ans++;
    return ans;
}

// LC 605: Can Place Flowers
bool canPlaceFlowers(vector<int>& bed, int n) {
    int count = 0, sz = bed.size();
    for (int i = 0; i < sz && count < n; i++) {
        if (bed[i] == 0 &&
            (i == 0 || bed[i-1] == 0) &&
            (i == sz-1 || bed[i+1] == 0)) {
            bed[i] = 1;
            count++;
        }
    }
    return count >= n;
}
```

Hash Map · 1 card

Hash Map — $O(1)$ Lookup

Maps keys to values using a hash function. Enables $O(1)$ average insert/lookup vs $O(n)$ linear search. Time $O(1)$ avg · Space $O(n)$

```
C++
// LC 1: Two Sum -  $O(n)$  with hash map
vector<int> twoSum(vector<int>& nums, int target) {
    unordered_map<int, int> mp; // value -> index
    for (int i = 0; i < nums.size(); i++) {
        int complement = target - nums[i];
        if (mp.count(complement))
            return {mp[complement], i};
        mp[nums[i]] = i;
    }
    return {};
}

// Frequency count
unordered_map<int, int> freq;
for (int x : nums) freq[x]++;
// Check if exists
if (freq.count(key)) { /* exists */ }
// Iterate
for (auto& [k, v] : freq) { /* use k, v */ }
```

```
C++
// Count pairs with difference == k
int countPairs(vector<int>& nums, int k) {
    int count = 0;
    for (int i = 0; i < nums.size(); i++)
        for (int j = i + 1; j < nums.size(); j++)
            if (abs(nums[i] - nums[j]) == k)
                count++;
    return count;
}

// Running sum (brute force  $O(n^2)$  - optimize with prefix sum)
vector<int> runningSum(vector<int>& nums) {
    int n = nums.size();
    vector<int> ans(n);
    for (int i = 0; i < n; i++) {
        int sum = 0;
        for (int j = 0; j <= i; j++) sum += nums[j];
        ans[i] = sum;
    }
    return ans;
}
```

Greedy · 1 card

Greedy — Locally Optimal Choices

Makes the best immediate choice at each step hoping to reach the global optimum. Simple but not always optimal.

Usually $O(n \log n)$ with sorting · Space $O(1)$

```
C++
// Max-heap (default)
priority_queue<int> max_pq;
max_pq.push(1);
int top = max_pq.top(); // 3
max_pq.pop();

// Min-heap
priority_queue<int, vector<int>, greater<int>> min_pq;

// Min-heap of pairs {cost, node}
priority_queue<pair<int,int>,
               vector<pair<int,int>>,
               greater<>> pq;
pq.push({cost, node});
auto [c, n] = pq.top(); pq.pop();
```

Kadane's Algorithm · 1 card

Maximum Subarray Sum

Finds the maximum sum of a contiguous subarray in $O(n)$. Resets the running sum to 0 whenever it goes negative.

Time $O(n)$ · Space $O(1)$

```
C++
// LC 53: Maximum Subarray
int maxSubArray(vector<int>& nums) {
    int globalSum = INT_MIN, localSum = 0;
    for (int x : nums) {
        localSum += x;
        globalSum = max(globalSum, localSum);
        if (localSum < 0) localSum = 0;
    }
    return globalSum;
}
```

```
Python
def maxSubArray(nums):
    global_sum = float('-inf')
    local_sum = 0
    for x in nums:
        local_sum += x
        global_sum = max(global_sum, local_sum)
        if local_sum < 0:
            local_sum = 0
    return global_sum
```

Linear Search · 1 card

Linear Search

Traverses every element checking against the target. Simplest search — no sorting required. $O(n)$ worst case.

Best $O(1)$ · Average/Worst $O(n)$

```
C++
// Generic linear search
int linearSearch(vector<int>& nums, int target) {
    for (int i = 0; i < nums.size(); i++)
        if (nums[i] == target) return i;
    return -1;
}

// LC 1295: Find Numbers with Even Digits
int findNumbers(vector<int>& nums) {
    int ans = 0;
    for (int x : nums) {
        int len = log10(x) + 1;
        if (len % 2 == 0) ans++;
    }
    return ans;
}
```

Linked List · 1 card

Linked List — Insert / Delete / Search

Nodes connected by next pointers. $O(1)$ insert/delete at head but $O(n)$ to access arbitrary elements. Lookup $O(n)$ · Insert/Delete head $O(1)$ · Insert/Delete tail $O(n)$

```
C++
struct ListNode {
    int val; ListNode* next;
    ListNode(int x) : val(x), next(nullptr) {}
};

// Insert at head
ListNode* insertHead(ListNode* head, int val) {
    ListNode* node = new ListNode(val);
    node->next = head;
    return node;
}

// Delete head
ListNode* deleteHead(ListNode* head) {
    if (!head) return nullptr;
    ListNode* tmp = head;
    head = head->next;
    delete tmp;
    return head;
}

// Floyd's cycle detection / find mid
ListNode* findMid(ListNode* head) {
    ListNode* slow = head, *fast = head;
    while (fast && fast->next) {
        slow = slow->next;
        fast = fast->next->next;
    }
    return slow;
}
```

MOD $(1e9 + 7)$ · 1 card

Modular Arithmetic

Apply % at each step to prevent overflow. $1e9+7$ is prime, enabling modular inverse via Fermat's Little Theorem.

$O(1)$ per operation · $O(\log n)$ for modular inverse

```
C++
const int MOD = 1e9 + 7;

// Correct: apply mod each step
long long factorial(int n) {
    long long fact = 1;
    for (int i = 2; i <= n; i++)
        fact = (fact * i) % MOD;
    return fact;
}

// Modular properties:
// (a + b) % m = (a%m + b%m) % m
// (a * b) % m = (a%m * b%m) % m
// (a - b) % m = (a%m - b%m + m) % m  - +m to avoid negatives

// Modular inverse (m must be prime): a^(m-2) mod m
long long power(long long base, long long exp, long long mod) {
    long long res = 1;
    base %= mod;
    while (exp > 0) {
        if (exp & 1) res = res * base % mod;
        base = base * base % mod;
        exp >>= 1;
    }
    return res;
}

long long modInverse(long long a) { return power(a, MOD - 2, MOD); }
```

Prefix Sum · 1 card

Prefix Sum — $O(1)$ Range Queries

Pre-computes cumulative sums to answer range sum queries in $O(1)$ after $O(n)$ build.

Build $O(n)$ · Query $O(1)$ · Space $O(n)$

```
C++
// Build prefix sum (0-indexed, padded)
vector<int> buildPrefix(vector<int>& a) {
    int n = a.size();
    vector<int> pref(n + 1, 0);
    for (int i = 0; i < n; i++)
        pref[i + 1] = pref[i] + a[i];
    return pref;
}

// Range sum [l, r] (0-indexed):
// pref[r + 1] - pref[l]

// LC 303: Range Sum Query
class NumArray {
public:
    vector<int> pref;

    NumArray(vector<int>& nums) {
        pref.resize(nums.size() + 1, 0);
        for (int i = 0; i < nums.size(); i++)
            pref[i + 1] = pref[i] + nums[i];
    }

    int sumRange(int l, int r) { return pref[r + 1] - pref[l]; }
};
```

Queue & Stack · 2 cards

Stack — LIFO

Last In First Out. Use for matching brackets, monotonic problems, undo operations.

Push/Pop $O(1)$

```
Python
# LC 20: Valid Parentheses
def isValid(s: str) -> bool:
    stack = []
    pairs = {'(': ')', '[': ']', '{': '}', '(': '}', '[': '}', '{': '}' }
    for c in s:
        if c in "([{":
            stack.append(c)
        elif not stack or stack[-1] != pairs[c]:
            return False
        else:
            stack.pop()
    return len(stack) == 0
```

```
C++
// Stack operations
stack<int> st;
st.push(1); st.push(2);
int top = st.top(); st.pop();
bool empty = st.empty();
```

Queue — FIFO

First In First Out. Use for BFS, level-order traversal, sliding window problems.

Push/Pop $O(1)$

```
Python
from collections import deque
queue = deque()
queue.appendleft(1) # enqueue front
queue.appendleft(2)
val = queue.pop() # dequeue back
```

```
C++
// Queue operations
queue<int> q;
q.push(1); q.push(2);
int front = q.front(); q.pop();
bool empty = q.empty();

// Deque (double-ended)
deque<int> dq;
dq.push_front(1); dq.push_back(2);
dq.pop_front(); dq.pop_back();
```

Sliding Window · 2 cards

Sliding Window — Fixed Size

Maintains a window of fixed size k , sliding one element at a time. Avoids recomputing from scratch each step.

Time $O(n)$ · Space $O(1)$

```
C++
// Maximum sum of subarray of size k
int maxSumSubarray(vector<int>& nums, int k) {
    int windowSum = 0, maxSum = 0;
    for (int i = 0; i < k; i++) windowSum += nums[i];
    maxSum = windowSum;
    for (int i = k; i < nums.size(); i++) {
        windowSum += nums[i] - nums[i - k];
        maxSum = max(maxSum, windowSum);
    }
    return maxSum;
}
```

Sliding Window — Variable Size

Expands right pointer and shrinks left pointer based on a condition. Common for longest/shortest subarray problems.

Time $O(n)$ · Space $O(k)$ with hash map

```
C++
// Longest substring without repeating characters
int lengthOfLongestSubstring(string s) {
    unordered_map<char, int> last;
    int ans = 0, l = 0;
    for (int r = 0; r < s.size(); r++) {
        if (last.count(s[r]) && last[s[r]] >= l)
            l = last[s[r]] + 1;
        last[s[r]] = r;
        ans = max(ans, r - l + 1);
    }
    return ans;
}
```

```
Python
def lengthOfLongestSubstring(s: str) -> int:
    last = {}
    ans = l = 0
    for r, c in enumerate(s):
        if c in last and last[c] >= l:
            l = last[c] + 1
        last[c] = r
        ans = max(ans, r - l + 1)
    return ans
```

Sorting · 4 cards

Built-in Sort — Usage & Comparators

C++ + std::sort is introsort ($O(n \log n)$). Python sort is Timsort (stable). Custom comparators allow multi-key sorting.

Time $O(n \log n)$ · Space $O(\log n)$

```
C++
// Sort ascending
sort(v.begin(), v.end());
// Descending
sort(v.begin(), v.end(), greater<int>());
// Custom: multi-key
sort(v.begin(), v.end(), [](const array<int, 3>& x, const array<int, 3>& y) {
    return x[0] != y[0] ? x[0] < y[0] :
           x[1] != y[1] ? x[1] < y[1] : x[2] < y[2];
});
```

Bubble Sort

Repeatedly swaps adjacent out-of-order elements. Early termination with a flag. Simple but $O(n^2)$.

Time $O(n^2)$ · Space $O(1)$

```

C++
void bubbleSort(vector<int>& arr) {
    int n = arr.size();
    bool swapped = true;
    for (int i = 0; i < n - 1 && swapped; i++) {
        swapped = false;
        for (int j = 0; j < n - i - 1; j++) {
            if (arr[j] > arr[j+1]) {
                swap(arr[j], arr[j+1]);
                swapped = true;
            }
        }
    }
}

```

Merge Sort

Divide array in half, recursively sort each half, merge. Stable, $O(n \log n)$ all cases.

Time $O(n \log n)$ all cases · Space $O(n)$

```

C++
void merge(vector<int>& a, int l, int m, int r) {
    vector<int> L(a.begin()+l, a.begin()+m+1);
    vector<int> R(a.begin()+m+1, a.begin()+r+1);
    int i=0, j=0, k=l;
    while (i<L.size() && j<R.size())
        a[k++] = (L[i]<=R[j]) ? L[i++] : R[j++];
    while (i<L.size()) a[k++] = L[i++];
    while (j<R.size()) a[k++] = R[j++];
}
void mergeSort(vector<int>& a, int l, int r) {
    if (l >= r) return;
    int m = l + (r-l)/2;
    mergeSort(a, l, m);
    mergeSort(a, m+1, r);
    merge(a, l, m, r);
}

```

Tim Sort

Hybrid of insertion sort + merge sort. Used in Python sort() and Java Arrays.sort(). Splits into runs of size 32, sorts each with insertion sort, then merges.

Time $O(n \log n)$ · Space $O(1)$

Two Pointers — Converging

Start one pointer at each end, move inward based on condition. Common for sorted arrays.

Time $O(n)$ · Space $O(1)$

```

C++
// LC 977: Squares of Sorted Array
vector<int> sortedSquares(vector<int>& nums) {
    int l = 0, r = nums.size() - 1;
    vector<int> res;
    while (l <= r) {
        if (abs(nums[l]) > abs(nums[r])) {
            res.push_back(nums[l]*nums[l]); l++;
        } else {
            res.push_back(nums[r]*nums[r]); r--;
        }
    }
    reverse(res.begin(), res.end());
    return res;
}

```

```

Python
def sortedSquares(nums):
    l, r = 0, len(nums) - 1
    res = []
    while l <= r:
        if abs(nums[l]) > abs(nums[r]):
            res.append(nums[l]**2); l += 1
        else:
            res.append(nums[r]**2); r -= 1
    return res[::-1]

```

```

C++
const int RUN = 32;

void insertionSort(vector<int>& nums, int l, int r) {
    for (int i = l+1; i <= r; i++) {
        int tmp = nums[i], j = i-1;
        while (j >= l && nums[j] > tmp) { nums[j+1]=nums[j]; j--; }
        nums[j+1] = tmp;
    }
}

```

```

void timMerge(vector<int>& nums, int l, int m, int r) {
    vector<int> L(nums.begin()+l, nums.begin()+m+1);
    vector<int> R(nums.begin()+m+1, nums.begin()+r+1);
    int i=0, j=0, k=l;
    while (i<L.size() && j<R.size()) nums[k++] = (L[i]<=R[j]) ? L[i++] : R[j++];
    while (i<L.size()) nums[k++] = L[i++];
    while (j<R.size()) nums[k++] = R[j++];
}

```

```

void timSort(vector<int>& nums) {
    int n = nums.size();
    for (int i=0; i<n; i+=RUN)
        insertionSort(nums, i, min(i+RUN-1, n-1));
    for (int sz=RUN; sz<n; sz+=2)
        for (int l=0; l<n; l+=2*sz) {
            int m=l+sz-1, r=min(l+2*sz-1, n-1);
            if (m<r) timMerge(nums, l, m, r);
        }
}

```

Time Complexity · 1 card

Input Size → Required Complexity

Use input size n to determine which algorithm complexity is fast enough. LeetCode typical time limit is ~10^8 operations/sec.

```

Reference
n ≤ 10      → O(n!), O(n^7), O(n^6)
n ≤ 20      → O(2^n)
n ≤ 80      → O(n^4)
n ≤ 400     → O(n^3)
n ≤ 7,500   → O(n^2)
n ≤ 10^6    → O(n log n), O(n)
n > 10^6    → O(log n), O(1)

```

```

// Loop complexities:
O(n)      ~ single loop
O(nm)     ~ nested loops
O(Vn)     ~ loop: i=1 <= n
O(log n)  ~ halving each step (binary search)
O(n log n)~ sort + single loop

```

Graph Theory

Tree Traversals · 1 card

Preorder / Inorder / Postorder

Three DFS traversal orders. Preorder: Root→L→R. Inorder: L→Root→R (sorted for BST). Postorder: L→R→Root.

```

C++
void preorder(TreeNode* node) {
    if (!node) return;
    // process node->val
    preorder(node->left);
    preorder(node->right);
}

```

```

void inorder(TreeNode* node) {
    if (!node) return;
    inorder(node->left);
    // process node->val
    inorder(node->right);
}

```

```

void postorder(TreeNode* node) {
    if (!node) return;
    postorder(node->left);
    postorder(node->right);
    // process node->val
}

```

```

Python
def preorder(node):
    if not node: return
    # process node.val
    preorder(node.left); preorder(node.right)

```

```

def inorder(node):
    if not node: return
    inorder(node.left)
    # process node.val
    inorder(node.right)

```

```

def postorder(node):
    if not node: return
    postorder(node.left); postorder(node.right)
    # process node.val

```

BFS · 1 card

Breadth-First Search

Trie · 1 card

Trie — Insert / Search / StartsWith

Prefix tree for string storage and retrieval. Each node has 26 children (a-z) and an end-of-word flag. $O(m)$ per operation where m = word length.

Time $O(m)$ per op · Space $O(26 \times n \times m)$

```

C++
struct TrieNode {
    TrieNode* children[26];
    bool isEnd;
    TrieNode() : isEnd(false) {
        fill(children, children+26, nullptr);
    }
};

class Trie {
public:
    TrieNode* root;
    Trie() { root = new TrieNode(); }

    void insert(string word) {
        TrieNode* cur = root;
        for (char c : word) {
            int i = c - 'a';
            if (!cur->children[i])
                cur->children[i] = new TrieNode();
            cur = cur->children[i];
        }
        cur->isEnd = true;
    }

    bool search(string word) {
        TrieNode* cur = root;
        for (char c : word) {
            int i = c - 'a';
            if (!cur->children[i]) return false;
            cur = cur->children[i];
        }
        return cur->isEnd;
    }

    bool startsWith(string prefix) {
        TrieNode* cur = root;
        for (char c : prefix) {
            int i = c - 'a';
            if (!cur->children[i]) return false;
            cur = cur->children[i];
        }
        return true;
    }
};

```

Two Pointers · 1 card

Explores all nodes at current depth before going deeper. Uses a queue. Guarantees shortest path in unweighted graphs.

Time $O(V+E)$ · Space $O(V)$

```

Python
def bfs(root, target):
    if not root: return None
    level = [root]
    while level:
        next_level = []
        for node in level:
            if not node: continue
            if node.val == target: return node
            next_level.append(node.left)
            next_level.append(node.right)
        level = next_level
    return None

```

```

C++
// BFS on graph
vector<int> bfs(vector<vector<int>>& adj, int src, int n) {
    vector<int> dist(n, -1);
    queue<int> q;
    dist[src] = 0; q.push(src);
    while (!q.empty()) {
        int u = q.front(); q.pop();
        for (int v : adj[u]) {
            if (dist[v] == -1) {
                dist[v] = dist[u] + 1;
                q.push(v);
            }
        }
    }
    return dist;
}

```

DFS · 1 card

Depth-First Search

Explores one branch completely before backtracking. Uses recursion (implicit stack). Great for path finding, cycle detection, connected components.

Time $O(V+E)$ · Space $O(V)$

```

Python
def dfs(node):
    if not node: return 0
    left = dfs(node.left)
    right = dfs(node.right)
    return max(left, right) + 1 # max depth example

```

```

C++
// DFS on graph with visited set
void dfs(vector<vector<int>>& adj, vector<bool>& vis, int u) {
    vis[u] = true;
    // process u
    for (int v : adj[u])
        if (!vis[v]) dfs(adj, vis, v);
}

```

Bellman-Ford · 1 card

Bellman-Ford Algorithm

Shortest paths from single source. Handles negative weights. Runs $|V| - 1$ relaxation passes over all edges.

Time $O(V \times E)$ · Space $O(V)$

```

C++
template<typename T, typename T_vector>
void bellman_ford(T, a3& g, T_vector& dist, int src, int mx_edges) {
    dist[src] = 0;
    for (int i = 0; i <= mx_edges; i++) {
        T_vector ndist = dist;
        for (auto x : g) {
            auto [from, to, cost] = x;
            ndist[to] = min(ndist[to], dist[from] + cost);
        }
        dist = ndist;
    }
}

```

```

Python
def bellman_ford(g, src, mx_edges):
    dist[src] = 0
    for _ in range(mx_edges + 1):
        ndist = dist[:]
        for fr, to, cost in g:
            ndist[to] = min(ndist[to], dist[fr] + cost)
        dist = ndist
    return dist

```

Dijkstra's · 1 card

Dijkstra's Algorithm

Shortest paths from single source. Non-negative weights only. Uses min-heap priority queue.

Time $O((V + E) \log V)$ · Space $O(V)$

```

C++
TreeNode* insert(TreeNode* root, int key) {
    if (!root) return new TreeNode(key);
    if (key < root->val) root->left = insert(root->left, key);
    else root->right = insert(root->right, key);
    return root;
}

TreeNode* search(TreeNode* root, int key) {
    if (!root || root->val == key) return root;
    return key < root->val ? search(root->left, key)
        : search(root->right, key);
}

TreeNode* deleteNode(TreeNode* root, int key) {
    if (!root) return nullptr;
    if (key < root->val) root->left = deleteNode(root->left, key);
    else if (key > root->val) root->right = deleteNode(root->right, key);
    else {
        if (!root->left) return root->right;
        if (!root->right) return root->left;
        TreeNode succ = root->right;
        while (succ->left) succ = succ->left;
        root->val = succ->val;
        root->right = deleteNode(root->right, succ->val);
    }
    return root;
}

```

Kruskal's MST · 1 card

Kruskal's Algorithm — Minimum Spanning Tree

Greedy MST: sort edges by weight, add edge if it doesn't form a cycle (using DSU). $O(E \log E)$.

Time $O(E \log E)$ · Space $O(V)$

```

C++
int kruskalMST(int n, vector<array<int,3>>& edges) {
    // edges: {weight, u, v}
    sort(edges.begin(), edges.end());
    DSU dsu(n);
    int total = 0;
    for (auto& [w, u, v] : edges)
        if (dsu.unite(u, v)) total += w;
    return total;
}

```

Tarjan's Algorithm · 1 card

Tarjan's — SCCs & Bridges

Finds Strongly Connected Components in $O(V + E)$. Also used to find bridges ($\text{low}[v] > \text{tin}[u]$) and articulation points.

```

C++
template<typename T, typename T_vector>
void dijkstra(T, pair& g, T_vector& dist, int start) {
    priority_queue<pair<int,int>, vector<pair<int,int>>, greater<>> pq;
    dist[start] = 0;
    pq.push({0, start});
    while (!pq.empty()) {
        auto [u, cost, u] = pq.top(); pq.pop();
        if (u, cost > dist[u]) continue;
        for (auto [v, w] : g[u]) {
            if (dist[v] > dist[u] + w) {
                dist[v] = dist[u] + w;
                pq.push({dist[v], v});
            }
        }
    }
}

```

DSU · 1 card

Disjoint Set Union (Union-Find)

Tracks connected components. Two operations: Find (get root) and Union (merge sets). Path compression + union by rank gives near $O(1)$ per op.

$O_\alpha(n)$ = $O(1)$ per op with optimizations

```

C++
class DSU {
public:
    vector<int> root, rank_;
    int cnt;
    DSU(int n) : root(n), rank_(n, 1), cnt(n) {
        iota(root.begin(), root.end(), 0);
    }
    int find(int x) {
        return x == root[x] ? x : root[x] = find(root[x]);
    }
    bool unite(int x, int y) {
        x = find(x); y = find(y);
        if (x == y) return false;
        if (rank_[x] < rank_[y]) swap(x, y);
        root[y] = x;
        if (rank_[x] == rank_[y]) rank_[x]++;
        cnt--;
        return true;
    }
    bool connected(int x, int y) { return find(x) == find(y); }
};

```

Topological Sort · 1 card

Topological Sort (Kahn's BFS)

Time $O(V + E)$ · Space $O(V)$

```

C++
struct Tarjan {
    int timer = 0;
    vector<int> tin, low;
    vector<bool> onStack;
    stack<int> stk;
    vector<vector<int>> sccs;

    void dfs(int u, vector<vector<int>>& g) {
        tin[u] = low[u] = timer++;
        stk.push(u); onStack[u] = true;
        for (int v : g[u]) {
            if (tin[v] == -1) { dfs(v, g); low[u] = min(low[u], low[v]); }
            else if (onStack[v]) low[u] = min(low[u], tin[v]);
        }
        if (low[u] == tin[u]) { // root of SCC
            vector<int> scc;
            while (stk.top() != u) {
                int v = stk.top(); stk.pop();
                onStack[v] = false; scc.push_back(v);
            }
            stk.pop(); onStack[u] = false; scc.push_back(u);
            sccs.push_back(scc);
        }
    }

    vector<vector<int>> run(int n, vector<vector<int>>& g) {
        tin.assign(n, -1); low.assign(n, -1); onStack.assign(n, false);
        for (int u = 0; u < n; ++u) if (tin[u] == -1) dfs(u, g);
        return sccs;
    }
};

```

LCA · 1 card

Lowest Common Ancestor

Deepest node that has both x and y as descendants. $\text{dist}(x,y) = \text{depth}(x) + \text{depth}(y) - 2 \cdot \text{depth}(\text{LCA})$.

Time $O(n)$ · Space $O(n)$

```

C++
// LC 236: LCA of Binary Tree
TreeNode* lowestCommonAncestor(TreeNode* root, TreeNode* p, TreeNode* q) {
    if (!root || !root->val) return nullptr;
    TreeNode* left = lowestCommonAncestor(root->left, p, q);
    TreeNode* right = lowestCommonAncestor(root->right, p, q);
    if (left && right) return root; // p and q on opposite sides
    return left ? left : right;
}

```

Linear ordering of a DAG where u comes before v for every edge (u,v). Detects cycles if not all nodes are processed.

Time $O(V + E)$ · Space $O(V)$

```

C++
struct TopologicalSort {
    vector<int> orders;
    bool isSorted = false;
    int steps = 0;

    TopologicalSort(vector<vector<int>>& g, vector<int>& indegree) {
        int n = g.size(); nodes = 0;
        queue<int> q;
        for (int i = 0; i < n; i++)
            if (indegree[i] == 0) q.push(i);
        while (!q.empty()) {
            int sz = q.size(); steps++;
            nodes += sz;
            for (int i = 0; i < sz; i++) {
                int u = q.front(); q.pop();
                orders.push_back(u);
                for (int v : g[u])
                    if (--indegree[v] == 0) q.push(v);
            }
            isSorted = nodes == n;
        }
    };

    // LC 207: Course Schedule
    bool canFinish(int n, vector<vector<int>>& pre) {
        vector<vector<int>> g(n); vector<int> ind(n);
        for (auto& p : pre) { g[p[1]].push_back(p[0]); ind[p[0]]++; }
        TopologicalSort ts(g, ind);
        return ts.isSorted;
    }
}

```

Binary Search Tree · 1 card

BST — Insert / Delete / Search

Left subtree < node < right subtree. All ops $O(n)$ worst case (unbalanced), $O(\log n)$ average.

Time $O(n)$ worst · $O(\log n)$ balanced

Math

Bit Manipulation · 2 cards

Bit Operations Cheat Sheet

Direct binary manipulation. All operations $O(1)$. Essential for subset enumeration and optimization.

```

C++
// Common bit tricks
n & (n-1) // remove lowest set bit (isPowerOfTwo: n>0 && !(n&(n-1)))
n & (-n) // isolate lowest set bit
n | (1<=i) // set bit i
n & ~(1<=i) // clear bit i
(n >> i) & 1 // get bit i
n ^ n = 0 // XOR with self cancels
n ^ 0 = n // XOR with 0 unchanged
__builtin_popcount(n) // count set bits
__builtin_ctz(n) // count trailing zeros
31 - __builtin_ctz(n) // index of highest set bit

// Enumerate all submasks of mask m:
for (int s = m; s; s = (s-1) & m) { /* use s */ }

// Subsets (bit masking):
for (int mask = 0; mask < (1<=n); mask++)
    for (int j = 0; j < n; j++)
        if ((mask >> j) & 1) /* j is in subset */ }

```

Bit Manipulation — Common Problems

XOR to find single number, Hamming weight, missing number, subsets.

```

C++
// Single Number (LC 136) - XOR cancels pairs
int singleNumber(vector<int>& nums) {
    int ans = 0;
    for (int x : nums) ans ^= x;
    return ans;
}

// Hamming Weight (LC 191)
int hammingWeight(uint32_t n) {
    int ans = 0;
    for (i; n; i &= n-1) ans++; // remove lowest set bit each time
    return ans;
}

// Missing Number (LC 268)
int missingNumber(vector<int>& nums) {
    int n = nums.size(); ans = n;
    for (int i = 0; i < n; i++) ans ^= i ^ nums[i];
    return ans;
}

```


Combinatorics · 1 card

Combinatorics — nCr with Modular Inverse

$C(n,r) = n! / (r!(n-r)!)$. Use modular inverse for large values. Pascal's Triangle gives $C(n,r) = C(n-1,r) + C(n-1,r-1)$.

Precompute $O(n)$ · Query $O(1)$

```
C++
struct Comb {
    int mod;
    vector<long long> fact, inv_fact;

    Comb(int N, int mod = 1e9+7) : mod(mod), fact(N+1), inv_fact(N+1) {
        fact[0] = 1;
        for (int i = 1; i <= N; i++) fact[i] = fact[i-1] * i % mod;
        inv_fact[N] = power(fact[N], mod-2);
        for (int i = N-1; i >= 0; i--) inv_fact[i] = inv_fact[i+1] * (i+1) % mod;
    }

    long long power(long long a, long long b) {
        long long res = 1; a %= mod;
        for (; b > 0; b >>= 1, a = a*a%mod)
            if (b&1) res = res*a%mod;
        return res;
    }

    long long nCr(int n, int r) {
        if (r < 0 || r > n) return 0;
        return fact[n] % mod * inv_fact[r] % mod * inv_fact[n-r] % mod;
    }
};
// Comb c(1000000); c.nCr(100, 5);
```

Matrix Exponentiation · 1 card

Matrix Exponentiation — $O(\log n)$ Recurrences

Compute linear recurrences (e.g., Fibonacci) in $O(k^3 \log n)$ by using binary exponentiation on the transformation matrix.

Time $O(k^3 \log n)$ · Space $O(k^2)$

A number has at most $\log(n)$ prime factors since $2^{\log(n)} \geq n$. Trial division up to $\sqrt{n}$ finds all factors. Time $O(\sqrt{n})$ · Space $O(\log n)$

```
C++
vector<int> primeFactors(int n) {
    vector<int> factors;
    for (int i = 2; i * i <= n; i++) {
        while (n % i == 0) {
            factors.push_back(i);
            n /= i;
        }
    }
    if (n > 1) factors.push_back(n);
    return factors;
}
```

Sieve — $O(\log n)$ Factorization per Query

Precompute minimum prime factor for every number $\leq$ MAX. Then factor any n in $O(\log n)$.

Sieve $O(MAX)$ · Factorize $O(\log n)$ · Space $O(MAX)$

```
C++
vector<int> buildSieve(int MAX) {
    vector<int> minP(MAX+1);
    iota(minP.begin(), minP.end(), 0);
    for (int i = 2; i*i <= MAX; i++)
        if (minP[i] == i) // i is prime
            for (int j = i*i; j <= MAX; j += i)
                minP[j] = min(minP[j], i);
    return minP;
}

vector<int> factorize(int n, vector<int>& minP) {
    vector<int> factors;
    while (n > 1) {
        factors.push_back(minP[n]);
        n /= minP[n];
    }
    return factors;
}
```

```
C++
const int MOD = 1e9+7;
using Mat = vector<vector<long long>>;

Mat multiply(const Mat& A, const Mat& B) {
    int n = A.size();
    Mat C(n, vector<long long>(n, 0));
    for (int i = 0; i < n; i++)
        for (int k = 0; k < n; k++) if (A[i][k])
            for (int j = 0; j < n; j++)
                C[i][j] = (C[i][j] + A[i][k] * B[k][j]) % MOD;
    return C;
}

Mat matpow(Mat A, long long p) {
    int n = A.size();
    Mat res(n, vector<long long>(n, 0));
    for (int i = 0; i < n; i++) res[i][i] = 1; // Identity
    for (; p > 0; p >>= 1, A = multiply(A, A))
        if (p & 1) res = multiply(res, A);
    return res;
}

// Fibonacci in O(log n)
long long fib(int n) {
    Mat T = {{1,1},{1,0}};
    Mat A = {{1},{0}};
    return multiply(matpow(T, n), A)[1][0];
}
```

Number Theory · 2 cards

Binary Exponentiation — $O(\log n)$ Powers

Efficiently computes a^n in $O(\log n)$ using the binary representation of the exponent. Each step squares the base and halves the exponent.

Time $O(\log n)$ · Space $O(1)$

Strings

Manacher's Algorithm · 1 card

Manacher's — $O(n)$ Palindrome Detection

Finds all palindromic substrings in $O(n)$ by inserting $\#$ separators and reusing previously computed palindrome lengths via mirroring.

Time $O(n)$ · Space $O(n)$

```
Python
def manachers(s: str) -> List[int]:
    # Insert # to unify odd/even cases
    t = '@' + '#'.join(s)
    n = len(t)
    p = [0] * n # palindrome radii
    c = r = 0
    for i in range(n):
        if i < r:
            p[i] = min(r - i, p[2*c - i])
            while i-p[i]-1 >= 0 and i+p[i]+1 < n and t[i-p[i]-1] == t[i+p[i]+1]:
                p[i] += 1
            if i + p[i] > r:
                c, r = i, i + p[i]
    return p # p[i] = radius of palindrome centered at t[i]
```

```
C++
string longestPalindrome(string s) {
    string t = "@#";
    for (char c : s) { t += c; t += '@'; }
    int n = t.size();
    vector<int> p(n, 0);
    int c = 0, r = 0;
    for (int i = 0; i < n; i++) {
        if (i < r) p[i] = min(r-i, p[2*c-i]);
        while (i-p[i]-1 >= 0 && i+p[i]+1 < n && t[i-p[i]-1] == t[i+p[i]+1]) p[i]++;
        if (i+p[i] > r) { c=i; r=i+p[i]; }
    }
    int best = max_element(p.begin(), p.end()) - p.begin();
    return s.substr((best - p[best])/2, p[best]);
}
```

Palindrome · 1 card

Palindrome Check — Multiple Methods

A string reads the same forwards and backwards. Check by comparing from both ends inward.

Time $O(n)$ · Space $O(1)$

```
C++
// Basic - computes base^exp
long long fastpow(long long base, long long exp) {
    long long res = 1;
    while (exp > 0) {
        if (exp & 1) res *= base;
        base *= base;
        exp >>= 1;
    }
    return res;
}

// With modulo - prevents overflow in modular arithmetic
long long modpow(long long base, long long exp, long long mod) {
    base %= mod;
    long long res = 1;
    while (exp > 0) {
        if (exp & 1) res = (res * base) % mod;
        base = (base * base) % mod;
        exp >>= 1;
    }
    return res;
}

// e.g., 3^13 mod 1e9+7
// modpow(3, 13, 1e9+7)
```

Sieve of Eratosthenes — All Primes $\leq N$

Finds all prime numbers up to N . Iteratively marks multiples of each prime as composite. Outer loop runs only to $\sqrt{N}$ — composites already marked by smaller primes.

Time $O(n \log \log n)$ · Space $O(n)$

```
C++
vector<bool> sieveOfEratosthenes(const int n) {
    vector<bool> isPrime(n + 1, true);
    isPrime[0] = isPrime[1] = false;
    for (int i = 2; i * i <= n; i++) {
        if (isPrime[i]) {
            for (int j = i * i; j <= n; j += i) {
                isPrime[j] = false;
            }
        }
    }
    return isPrime;
}

// Usage: count primes up to 10^6
// auto p = sieveOfEratosthenes(1e6);
// count(p.begin(), p.end(), true) ~ 78498
```

Prime Factors · 2 cards

Prime Factorization — Trial Division

```
C++
// Method 1: Two pointers
bool isPalindrome(string s) {
    int l = 0, r = s.size() - 1;
    while (l < r) {
        if (s[l] != s[r]) return false;
        l++; r--;
    }
    return true;
}

// Method 2: Reverse compare
bool isPalindrome2(string s) {
    string rev = s;
    reverse(rev.begin(), rev.end());
    return s == rev;
}

// Method 3: STL equal with reverse iterators
bool isPalindrome3(string s) {
    return equal(s.begin(), s.begin() + s.size()/2, s.rbegin());
}
```

```
Python
# All in one line
def isPalindrome(s: str) -> bool:
    return s == s[::-1]

# Two pointers
def isPalindrome2(s: str) -> bool:
    l, r = 0, len(s) - 1
    while l < r:
        if s[l] != s[r]: return False
        l += 1; r -= 1
    return True
```

Z Algorithm · 1 card

Z Algorithm — $O(n)$ Pattern Matching

Builds Z-array where $Z[i]$ = length of longest substring starting at i matching a prefix. $Z[0] = 0$ by convention.

Time $O(n)$ · Space $O(n)$

```

C++
vector<int> zFunction(string s) {
    int n = s.size();
    vector<int> z(n, 0);
    int l = 0, r = 0;
    for (int i = 1; i < n; i++) {
        if (l < r) z[i] = min(r - i, z[i - l]);
        while (i + z[i] < n && s[i] == s[i + z[i]]) z[i]++;
        if (i + z[i] > r) { l = i; r = i + z[i]; }
    }
    return z;
}

// Pattern matching: find all occurrences of pattern in text
// Concat: pattern + 's' + text, check z[i] == pattern.size()
vector<int> findPattern(string pat, string txt) {
    string s = pat + 's' + txt;
    auto z = zFunction(s);
    vector<int> matches;
    int m = pat.size();
    for (int i = m + 1; i < s.size(); i++)
        if (z[i] == m) matches.push_back(i - m - 1);
    return matches;
}

```

```

Python
def z_function(s: str) -> list[int]:
    n = len(s)
    z = [0] * n
    l = r = 0
    for i in range(1, n):
        if i < r:
            z[i] = min(r - i, z[i - l])
            while i + z[i] < n and s[z[i]] == s[i + z[i]]:
                z[i] += 1
            if i + z[i] > r:
                l, r = i, i + z[i]
    return z

```